

LAB NO 6

DATA STRUCTURES AND ALGORITHMS

OBJECTIVE: To find an element in linear array using Linear Search and Binary Search.

LAB TASKS

1. Declare an array of size 10 to store account balances. Initialize with values 0 to 1000000. Check all array if any value is less than 10000. Show message:

Account No. Low Balance

Account No. Low Balance

Input 01:

```
class Account {
    int accountNo;
    int balance;
    public Account(int acc, int bal) {
        this.accountNo = acc;
        this.balance = bal;
    }
    // Overriding toString for OUTPUT
    @Override
    public String toString() {
        return "Account_No: " + accountNo + ", Balance: " + balance;
    }
}

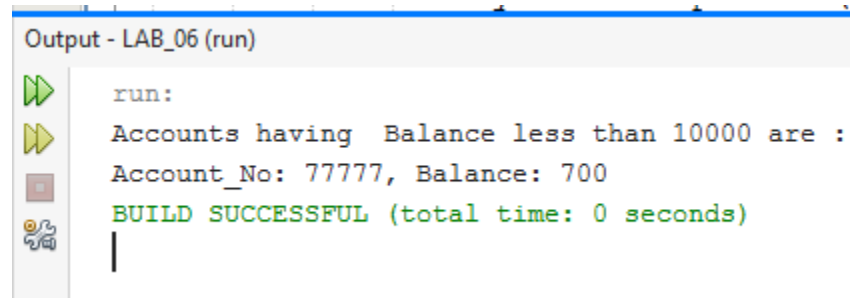
public class LAB_06 {
    // Print array of accounts
    public static void printArr(Account[] arr) {
        for (Account account : arr) {
            System.out.println(account);
        }
    }

    public static void main(String[] args) {
        Account[] accounts = {
            new Account(302004, 80500),

```

```
public static void main(String[] args) {  
    Account[] accounts = {  
        new Account(302004, 80500),  
        new Account(11994, 16050),  
        new Account(73958, 24500),  
        new Account(80414, 55200),  
        new Account(77777, 700)  
    };  
    int limit = 10000;  
    System.out.println("Accounts having Balance less than " + limit + " are :");  
    linearSearch(accounts, limit);  
}  
  
// Linear search method  
public static void linearSearch(Account[] accounts, int limit) {  
    for (Account acc : accounts) {  
        if (acc.balance < limit) {  
            System.out.println(acc);  
        }  
    }  
}
```

Output 01:



```
Output - LAB_06 (run)  
  
run:  
Accounts having Balance less than 10000 are :  
Account_No: 77777, Balance: 700  
BUILD SUCCESSFUL (total time: 0 seconds)
```

2. Write a program to search in array using Array built-in class.

Input 02:

```
public static void main(String[] args) {  
    int[] sArray = {45, 17, 80, 60, 129, 904};  
    System.out.println("Original Array: " + Arrays.toString(sArray));  
    Arrays.sort(sArray);  
    System.out.println("Sorting the Array : " + Arrays.toString(sArray));  
    int fetch = 80;  
    System.out.println("Searching : " + fetch);  
  
    int i = Arrays.binarySearch(sArray, fetch);  
    if (i >= 0) {  
        System.out.println("Element is " + fetch + " found after sorting: " + (i));  
    } else {  
        System.out.println("Element " + fetch + " not found ");  
    }  
}
```

Output 02:

```
Output - LAB_06 (run)  
  
run:  
Original Array: [45, 17, 80, 60, 129, 904]  
Sorting the Array : [17, 45, 60, 80, 129, 904]  
Searching : 80  
Element is 80 found after sorting: 3  
BUILD SUCCESSFUL (total time: 0 seconds)  
|
```

3. Given an unsorted array arr of integers, find the smallest positive integer that is **missing** from the array. You need to implement this using **binary search**. The array can contain both negative numbers and positive numbers, and you can assume that the array does not have duplicates.

Input 03:

```
//public class LAB_06
public static int findMissingPositive(int[] arr) {
    //Removing -ve No's as that they are not needed
    int n = arr.length;
    int[] positiveArr = new int[n]; //Using another array to ensure +ve values
    int index = 0;
    for (int elem : arr) {
        if (elem > 0) {
            positiveArr[index++] = elem;
        }
    }
    arr = Arrays.copyOf(positiveArr, index); //resizing arr by copied array
    Arrays.sort(arr); // Sort the array

    //Binary Search for Missing Number
    int left = 0, right = arr.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        int expected = mid + 1;

        if (arr[mid] == expected) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return left + 1;
}

public static void main(String[] args) {
    int[] arr = {8, 2, 4, -3, -1, 7, 4, 5, -9, 7};
    System.out.println("the smallest positive integer is " + findMissingPositive(arr));
}
```

Output 03:

```
Output - LAB_06 (run)

run:
the smallest positive integer is 1
BUILD SUCCESSFUL (total time: 0 seconds)
```

4. You are given a sorted array `arr[]` and a target element `target`. Your task is to find the first occurrence of the target in the array using binary search. If the target is not found, return -1. You are given a sorted array `arr[]` and a target element `target`. Your task is to find the first occurrence of the target in the array using binary search. If the target is not found, return -1

Input 04:

```
public static int findTarget(int[] arr,int target) {  
    int left = 0, right = arr.length - 1;  
    System.out.println("applying Binary Searching Technique:");  
    while (left <= right) {  
        int mid = left + (right - left) / 2;  
        System.out.println("Left: " + left + ", Right: " + right + ", Mid: " + mid + ", Value at Mid: " + arr[mid]);  
        if (arr[mid] == target) {  
            System.out.println("Target found at index " + mid);  
            return mid;  
        } else if (arr[mid] < target) {  
            left = mid + 1;  
        } else {  
            right = mid - 1;  
        }  
    }  
    System.out.println("Target not found");  
    return -1;  
}  
  
public static void main(String[] args) {  
    int[] arr = {13,20,46,25,64,757,23,7,131,13};  
    Arrays.sort(arr);  
    int target=9;  
    System.out.println("Postion of Target: "+target+" is: " + findTarget(arr,target)+" in Sorted Array "); // Output
```

Output 4:

```
Output - LAB_06 (run)  
run:  
applying Binary Searching Technique:  
Left: 0, Right: 9, Mid: 4, Value at Mid: 23  
Left: 0, Right: 3, Mid: 1, Value at Mid: 13  
Left: 0, Right: 0, Mid: 0, Value at Mid: 7  
Target not found  
Postion of Target: 9 is: -1 in Sorted Array  
BUILD SUCCESSFUL (total time: 0 seconds)
```

HOME TASKS:

5. Write a program initializing array of size 20 and search an element using binary search

Input 01:

```
        if (result != -1) {
            System.out.println(" The target " + target + " is found at index: " + result);
        } else {
            System.out.println(" The target " + target + " is not found in the array.");
        }
    }

    public static int binarySearch(int[] arr20, int target) {
        int lb = 0, ub = arr20.length - 1;

        while (lb <= ub) {
            int mid = (lb + ub) / 2;
            if (arr20[mid] == target) {
                return mid;
            } else if (arr20[mid] < target) {
                lb = mid + 1;
            } else {
                ub = mid - 1;
            }
        }

        return -1;
    }

    public static void printArray(int[] arr) {
        for (int num : arr) {
            System.out.print(num + " ");
        }
        System.out.println();
    }
```

Output 1:

```
Output - LAB_06 (run)

run:
Original Array:
18 37 78 36 61 97 60 13 45 44 8 63 92 0 98 51 27 26 56 34

Sorted Array:
0 8 13 18 26 27 34 36 37 44 45 51 56 60 61 63 78 92 97 98

Enter an integer to search (0-101):
92
The target 92 is found at index: 17
BUILD SUCCESSFUL (total time: 22 seconds)
```

6. Write a function called `occurrences` that, given an array of numbers `A`, prints all the distinct values in `A` each followed by its number of occurrences.

For example, if `A = (28, 1, 0, 1, 0, 3, 4, 0, 0, 3)`, the function should output the following five lines (here separated by a semicolon) `"28 1; 1 2; 0 4; 3 2; 4 1"`.

Input 02:

```
public static void main(String[] args) {  
    int[] A = {70,3,12,13,21,3,12,3,70,13,13,21};  
    occurrences(A);  
}  
  
public static void occurrences(int[] A) {  
    // Create a HashMap to store the number and its occurrences  
    Map<Integer, Integer> map = new HashMap<>();  
    // Loop through the array and count occurrences  
    for (int num : A) {  
        map.put(num, map.getOrDefault(num, 0) + 1);  
    }  
    for (Integer key : map.keySet()) {  
        int value = map.get(key);  
        System.out.print(key + " " + value+";");  
    }System.out.println();  
}
```

Output 2:

```
Output - LAB_06 (run)  
  
run:  
3 3;21 2;70 2;12 2;13 3;  
BUILD SUCCESSFUL (total time: 0 seconds)
```

Input 03:

```

    for (int i = index; i >= 0; i--) {
        if (arr[i].balance < threshold) {
            System.out.println(arr[i]);
        } else {
            break;
        }
    }
}

public static void main(String[] args) {
    Accounts[] accounts = {
        new Accounts(101, 2500),
        new Accounts(102, 1500),
        new Accounts(103, 8000),
        new Accounts(104, 500),
        new Accounts(105, 1200),
        new Accounts(106, 700)
    };

    int threshold = 1000;
    // Function to show accounts below the threshold
    fAccBeloThreshold(accounts, threshold);
}

```

```
class Accounts {
    int accountNo;
    int balance;

    public Accounts(int accountNo, int balance) {
        this.accountNo = accountNo;
        this.balance = balance;
    }

    @Override
    public String toString() {
        return "Account No: " + accountNo + ", Balance: " + balance;
    }
}

class BankSystem {
    // Binary search to find the first account below the threshold
    public static int binarySearch(Accounts[] arr, int threshold) {
        int lb = 0, ub = arr.length - 1;
        int result = -1;

        while (lb <= ub) {
            int mid = lb + (ub - lb) / 2;

            if (arr[mid].balance < threshold) {
                result = mid; // Store the index in case it's the last valid one
                ub = mid - 1; // Continue searching lower to find the first occurrence
            } else {
                lb = mid + 1;
            }
        }
        return result;
    }

    // Method to find and display accounts with balances below the threshold
    public static void fAccBeloThreshold(Accounts[] arr, int threshold) {
        // Sort the array based on balance
        Arrays.sort(arr, (a, b) -> Integer.compare(a.balance, b.balance));

        System.out.println(" Searching for accounts with balance below: " + threshold);
        int index = binarySearch(arr, threshold);

        if (index == -1) {
            System.out.println(" No accounts found with balance below the threshold.");
        } else {
            System.out.println(" Accounts below threshold:");
            for (int i = index; i >= 0; i--) { // Traverse backward for all valid accounts

```

Output 3:

```
Output - LAB_06 (run)
run:
  Searching for accounts with balance below: 1000
  No accounts found with balance below the threshold.
BUILD SUCCESSFUL (total time: 0 seconds)
|
```